个人项目作品集 • 2026.10

校园网络数字孪生运维平台

真实网络数据 · LangGraph 多 Agent · MCP 双向集成

5 设备 / 6 主机 真实纳管

38 场景评测 100%

171 + 277 tests

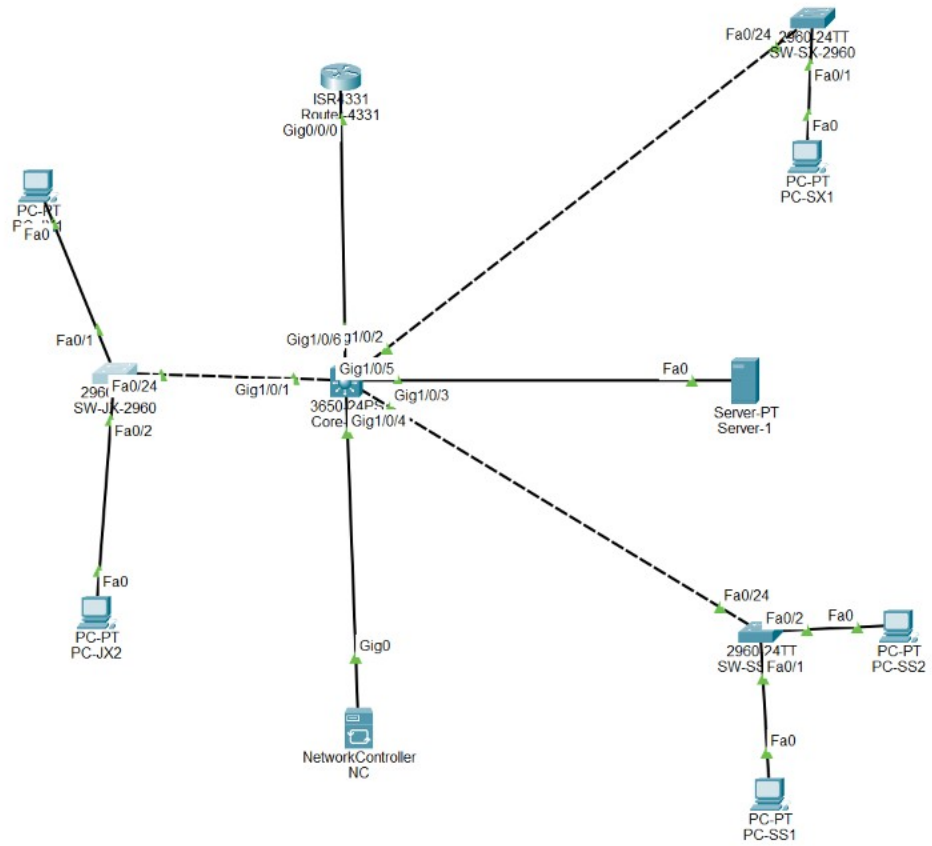
MCP 双向集成

从真实网络采集、故障检测、 AI 归因到人机确认与 MCP 工具协议化的完整闭环

截图均为真实运行（3 张 Mock 状态图已在正文标注）

01 · 实验环境： Packet Tracer 9 + Network Controller

拓扑 / 端口 / 可达性由 NC REST API 实时采集，不是静态示意图



环境要点

· 5 台网络设备全部被 NC 纳管（Managed）：Router-4331 / Core-3650 / 3×2960

· 6 台 PC + Server-1；拓扑 5 节点 4 链路，端口级采集

· Network Controller 提供 Real World Access REST API （本项目唯一真实数据源）

· IOS-XE 16.x / IOS 15.0； VLAN 10/20/30/40/50 + Trunk + OSPF

数据链路： NC REST API → 后端 5s 采集 → WebSocket → 3D 大屏

02 · 系统架构：真实数据 + LangGraph 多 Agent + MCP

全部组件在本机真实运行； 7 个进程协同



端口： PT NC 58000 · 后端 8000 · Agent 8100 · 前端 5173 · MCP Bridge 8200 · MCP Inspector 6274 · Webhook 9000

03 · 3D 数字孪生大屏（真实数据）

左侧：平台总览；右侧：点击设备查看真实 NC 数据



平台总览：数据来源「PT NC 真实采集」、5/5 设备在线、采集状态 netcon 正常、WebSocket 已连接



设备详情：型号、管理 IP、可达性、端口状态与关联主机全部来自 NC API

· 3D 场景为配置驱动：楼栋 / 设备 / 链路坐标来自场景配置，新增设备自动出现（pending → 确认 / 撤销）

· 告警脉冲、数据新鲜度、WebSocket 实时推送：故障注入实测 5 秒内大屏更新

04 · 告警生命周期与场景自动化

左：告警删除 / 清空（软删除 + 审计 + 撤销）；右： PT 加设备自动发现



活跃告警禁止删除只能静默；已恢复告警可单条删除 / 一键清空， 10 秒内可撤销并写审计

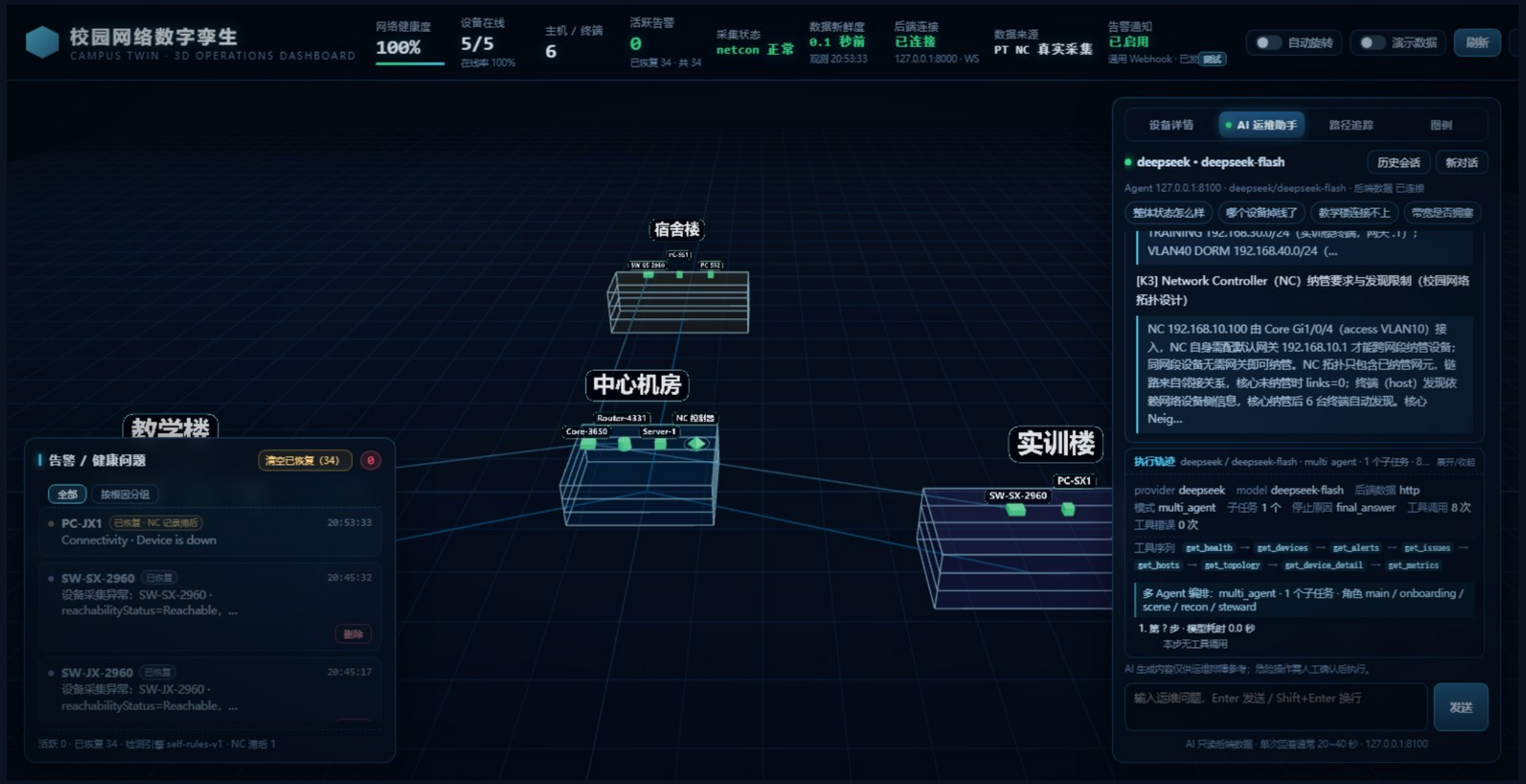


PT 加设备 → 采集发现 → 侦察 Agent 推断归属楼栋 → 场景自动出现（此张为 Mock 状态图 ⚠）

· 告警恢复判定不依赖 NC （NC 记录滞后 >3 分钟）：用主机 ping / 设备可达性独立判定，标记 stale_source
· 删除 / 清空类写操作全部走「用户确认」： Agent 只生成 pending action ，不直接执行

05 · AI 多 Agent 执行轨迹（LangGraph）

一次提问的完整执行链：意图 → 子任务计划 → 工具序列 → 汇总



LangGraph StateGraph：意图分类（规则优先 + LLM）→ 8 个 role 节点并行 → 汇总；trace 记录 plan、工具调用、耗时与停止原因

06 · 危险操作人机确认（interrupt）

写操作在图中暂停，用户确认后才执行；取消则数据不变



LangGraph interrupt() 暂停图 → 前端红色确认卡片 → 取消后 38 条已恢复告警一条未动（实测）； action 10 分钟 TTL + CAS + 审计

07 · 路径追踪 3D （真实 NC flow-analysis ）

PC-JX1 → SW-JX-2960 → Core-3650 → Server-1 ： 4 跳，约 1 秒



调用 NC flow-analysis 真实接口； PT 只返回部分中间设备 → 用拓扑 + 主机接入关系 BFS 补全中间 hop （标 synthetic ）， 3D 发光路径 + 逐跳端口 + 断点定位

08 · 断点定位与告警根因分组

左：链路中断时路径断点；右：根因聚合并抑制下游告警



路径 partial 时最后一跳红色 ✕ + 红色末段 + 面板断点原因（此张为 Mock 状态图 ⚠）



根因组「上联链路中断」：成员 3、被抑制 2；下游告警灰显「被上游抑制」（此张为 Mock 状态图 ⚠）

- 根因优先级：link_down > device_unreachable > device_offline > host_down
- 抑制仅展示层：不删除告警、不改状态，避免「告警风暴」淹没根因

09 • Agent 通过 MCP 读取真实配置文件

Agent → MCP Bridge(8200) → filesystem MCP Server(stdio) → 真实文件



trace.mcp_calls： filesystem/read_text_file ok · 6.88ms；回答引用真实 VLAN 10/20/30/40/50 与 trunk 配置；只读白名单，写操作被拒绝

10 · AI 流式输出与 RAG 引用

左： SSE 逐 token 渲染中；右：检索增强 + [K1]~[K5] 引用



SSE 流式中：状态条「正在检索知识库（命中 5 条）」 + 工具 chip + 停止生成按钮



RAG： 18 个知识块 BM25 检索命中 5 条，回答带 [K1]~[K5] 可点击引用与依据

- 真实 LLM 经 LangChain model.stream 推流，实测单次 2600+ token 事件；前端 80ms 节流渲染 + 停止生成 + 自动回退
- 无命中时明确回答「知识库无相关内容」，不编造

11 · MCP 双向集成能力清单

A： 对外 MCP Server （10 工具 /4 资源 /3 提示词）； B： 对内 MCP Client 桥接（23 工具）



官方 SDK 需 Python≥3.10 → MCP 进程独立 Python 3.14 环境，主服务 3.9 零侵入；只读白名单 + localhost + 全量审计日志

12· 真实测试与评测数据

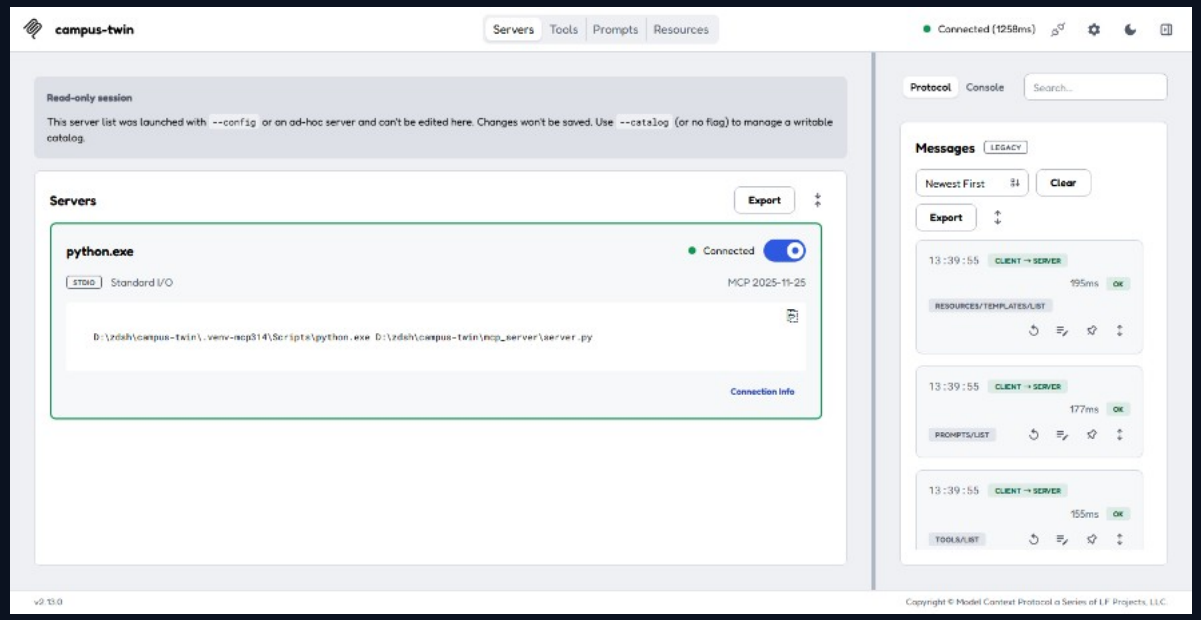
后端 171 / Agent 277 / MCP 30 tests； 38 场景离线评测 100%



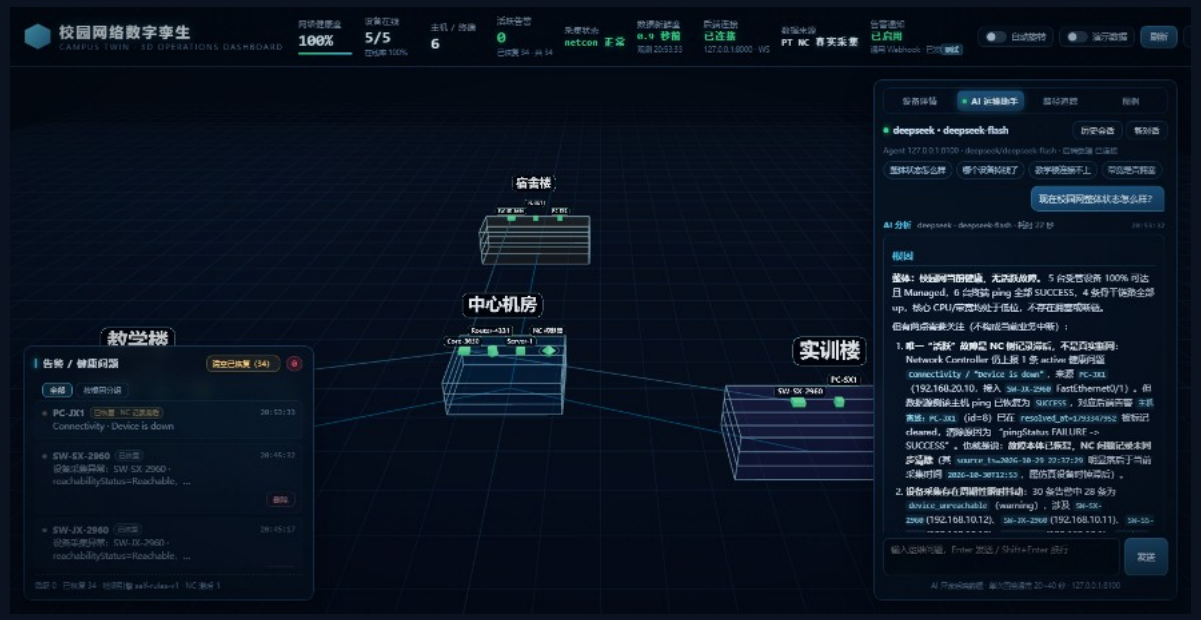
自动执行违规 0 （所有写操作必须人工确认）；评测集覆盖 17 类场景；全部离线可复现

13 · MCP 协议验证与真实 AI 排障

左： MCP Inspector 连接成功；右：真实 DeepSeek 排障回答



MCP Inspector： campus-twin Connected (1258ms)，协议 MCP 2025-11-25，TOOLS/PROMPTS/RESOURCES 列表全部 OK



AI 真实排障：根因、依据（设备 /IP/ 端口 / 时间）、处置建议；危险操作走确认卡片

· 任意 MCP 客户端（Claude Desktop / Cursor / VS Code / Inspector）可接入平台能力

· AI 回答强制引用真实数据；无数据明确说明，不编造

14 • RAG 评测闭环（W15）

56 篇文档 / 107 chunk / 3 种切分 / 48 query / 4 种检索配置



口径： recall@5 = top-5 命中相关块的比例； MRR = 首个相关结果排名倒数的均值；引用准确率 = 引用块中真正含答案证据的比例（严格口径）； faithfulness = 回答断言被引用内容支持的比例；可溯源率 = 引用均可定位到检索结果。

最佳配置： semantic 切分 + hybrid(RRF)， 48 query；另附 3 种切分 × 4 种检索配置的完整对比（含 rerank）。

15 · 可靠性 / 可观测性 / 可部署性（W15）

故障注入 · 断点恢复 · trace API · Docker + GitHub Actions CI

Agent 可靠性 + 可观测性 + 可部署性（真实测试/CI）

故障注入 · 重试/熔断/幂等 · 断点恢复 · trace API · Docker/CI

A. 可靠性实验（本地故障注入）

• 500 / 超时	指数退避重试 2 次（1s/3s），4xx 不重试
• 熔断	连续失败 5 次 → OPEN 60s，冷却后 HALF_OPEN 恢复
• 幂等	同请求窗口内缓存命中，HTTP 只打 1 次
• 脏数据	脏 JSON / 超大响应 / 类型错误 → ok=false 不抛栈
• HITL	写操作必须人工确认，自动执行违规 0
• 测试规模	Agent 277 passed；38 场景双引擎 100%

B. 断点恢复（进程重启后继续）

场景	写操作 → interrupt 暂停 → 进程退出 → 同会话恢复
验证	paused_before=None / paused_after=None
结果	resumed=None / executed=None / 总耗时 4030.45 ms
进程	start pid=None → resume pid=None
机制	LangGraph SqliteSaver checkpointer（thread_id=conversation_id）

C. 可观测性（自建 trace，替代 Langfuse）

· trace_events 表：step/tool/provider/model/tokens/耗时/ok

· API：/api/traces · /api/traces/{id} · /api/metrics/agent · /api/rag/report

· 前端「可观测性」页签：指标卡 + trace 列表/展开 + RAG 指标

· token 无 usage 时按 char/4 估算并标注 estimated

D. 可部署性（Docker 真实运行 + CI 全绿）

· WSL2 内安装 Docker Engine 28.1.1 + Compose v2.35.1

· docker compose config 真实校验 PASS（4 服务）

· hello-world 容器真实运行成功（镜像源 docker.1panel.live）

· GitHub Actions CI：6 个离线 job 全绿（1m23s）

技术栈与核心数据

全部真实运行，可现场演示 / 复现

技术栈

Python / FastAPI / asyncio / SQLite WAL
LangGraph / LangChain Core / langchain-openai
MCP （Server + Client 桥接）
SSE / WebSocket / RAG （BM25-lite）
Three.js / Vite / 原生 JS
Cisco Packet Tracer 9 / NC REST API
Cisco IOS-XE （3650 / 2960 / ISR4331）
DeepSeek / OpenAI 兼容 API

说明：截图均为真实运行； 04/08/09 三张为 Mock 状态图，用于展示需故障注入的 UI 状态。

核心数据

真实网络	**5 设备 / 6 主机 / 4 链路**
测试	**171 + 277 + 30 tests**
评测	**38 场景离线：根因 / 设备 / 动作 100%**
安全	**自动执行违规 0**
AI	**4 节点 + 8 role / 23 工具**
MCP	**Server 10 工具 + Bridge 23 工具**
前端	**3D 大屏 / 流式 / 引用**

个人项目 · 独立开发 · 2026.10